

Notice technique — de chaque diagramme au code

Explication exhaustive, élément par élément, des six diagrammes de `EBIOS-RM-diagrammes-UML.pdf` : ce que chaque boîte, chaque flèche et chaque notation désigne précisément dans le code source, avec le fichier concerné. Aucune analogie. Là où le diagramme représente une intention non encore codée, ce document le dit explicitement plutôt que de laisser croire à une correspondance qui n'existe pas.

SOMMAIRE

- | | |
|--|--|
| 01 Cas d'utilisation | 04 Séquence — chaque message tracé |
| 02a Vue d'ensemble — les liens en pointillés | 05 États — chaque transition, chaque méthode |
| 02b Classes — chaque relation, une par une | 06 Déploiement |
| 03 Composants — chaque bloc, mappé au code | |

01 Diagramme de cas d'utilisation

Trois acteurs, onze cas d'usage, une relation `«include»`. Chaque acteur est un rôle applicatif — au sens strict, une valeur d'énumération de rôle qui n'existe même pas encore dans le code, puisqu'aucune authentification n'est implémentée (voir §03). Le diagramme décrit donc une répartition de responsabilités *prévue*, pas un contrôle d'accès en vigueur : aujourd'hui, n'importe quel appelant HTTP peut exécuter n'importe quel cas d'usage.

Analyste risque — UC1, UC2, UC3, UC7, UC10

- UC1 Créer une étude — `RÉEL` `POST /api/v1/etudes`, géré par l'endpoint `app.MapPost("/api/v1/etudes", ...)` dans `Program.cs`, qui appelle `Etude.Creer(nom, perimetre,`

mission) puis `IEtudeRepository.AjouterAsync`.

- UC2 Saisir les données d'un atelier — **RÉEL** désigne collectivement l'ensemble des endpoints de création/modification par atelier (par exemple `POST .../valeurs-metier`, `PUT .../scenarios-de-risque/{id}/risque-residuel`). Il n'existe pas un seul endpoint « saisir » : ce cas d'usage regroupe environ 70 endpoints CRUD distincts dans `Program.cs`.
- UC3 Démarrer un atelier — **RÉEL** `POST /api/v1/etudes/{id}/demarrer-atelierN` ($N = 1 \text{ à } 5$), chacun appelant `Etude.DemarrerAtelierN()`.
- UC7 Consulter l'historique des versions — **PARTIELLEMENT RÉEL** chaque `SnapshotAtelier` porte un numéro de version croissant et n'est jamais supprimé (table `core_engine.snapshots_atelier`), donc l'historique existe en base. Mais aucun endpoint ne liste les versions passées : `ISnapshotAtelierRepository` n'expose que `ObtenirDernierParEtudeIdAsync` (le dernier) et `CompterParEtudeIdAsync` (un compte). Consulter une version *antérieure* à la dernière n'est possible qu'en lisant directement la table.
- UC10 Demander une suggestion IA — **NON IMPLÉMENTÉ** aucun code ne correspond à ce cas d'usage. Aucun dossier `Modules/IA` ou équivalent n'existe dans le projet.

Responsable / Validateur — UC4, UC5, UC6, UC7, UC11

- UC4 Valider un atelier — **RÉEL** `POST /api/v1/etudes/{id}/valider-atelierN`. C'est le flux détaillé dans le diagramme de séquence (§04).
- UC5 Rouvrir un atelier validé — **RÉEL** `POST /api/v1/etudes/{id}/rouvrir-atelierN`, appelant `Etude.RouvrirAtelierN()`.
- UC6 Générer un rapport — **RÉEL** `GET /api/v1/etudes/{id}/rapports/atelierN` ($N=1..5$) et `GET /api/v1/etudes/{id}/rapports/synthese`, définis dans `Program.cs` et implémentés par les classes de `Modules/Reporting/`.
- UC11 Accepter / rejeter une suggestion — **NON IMPLÉMENTÉ**, lié à UC10.

Administrateur — UC8, UC9

- UC8 Gérer le référentiel — **RÉEL** mais partiel : `POST/PUT/DELETE .../socle-securite/referentiels` gère `ReferentielApplicable` (Atelier 1) et le catalogue ISO 27001 statique de `frontend/src/lib/iso27001.ts` — mais ce catalogue est codé en dur dans le frontend, il n'existe aucun endpoint pour l'administrer côté serveur.
- UC9 Gérer comptes et rôles — **NON IMPLÉMENTÉ**. Aucune table utilisateur, aucun modèle de rôle en base.

La relation «include»

UC10 -.-> UC11 : dans la notation UML, une flèche pointillée étiquetée «include» signifie que le cas d'usage source (UC10) exécute systématiquement le cas d'usage cible (UC11) comme sous-étape obligatoire — demander une suggestion implique toujours,

ensuite, de devoir l'accepter ou la rejeter. Cette relation est purement descriptive du cas d'usage prévu ; comme UC10 et UC11 ne sont pas implémentés, elle n'a aucune contrepartie dans le code actuel.

02a Vue d'ensemble — les liens en pointillés

Ce diagramme a cinq flèches pleines (le flux normal, atelier après atelier) et deux flèches en pointillés. Les pointillés signalent une différence de nature, pas seulement de style : une flèche pleine représente une *dépendance de données à la création* (l'atelier cible lit des données produites par l'atelier source), une flèche pointillée représente un *chemin de réouverture* — un appel possible après coup, qui n'engendre aucune donnée nouvelle par lui-même.

A5 --> A1 : cycle stratégique complet

Concrètement : une fois l'Atelier 5 validé, l'utilisateur peut appeler `POST /api/v1/etudes/{id}/rouvrir-atelier1`, qui exécute `Etude.RouvrirAtelier1()` (fichier `Domain/Cadrage/Etude.cs`) et fait repasser `Statut de Validee` à `EnCours`. L'analyste peut alors modifier les valeurs métier, biens support ou événements redoutés, puis revalider l'atelier 1 — ce qui crée une nouvelle version de `SnapshotAtelier` (numéro d'atelier 1, version incrémentée). Le terme « cycle stratégique complet » est une étiquette méthodologique (issue de la documentation ANSSI) apposée sur ce diagramme : **aucune méthode ni endpoint du code ne s'appelle « cycle stratégique »**, et rouvrir l'atelier 1 ne rouvre pas automatiquement les ateliers 2 à 5 — chacun garde son propre statut indépendant (`StatutAtelier2`, etc., restent `Validee` tant qu'ils ne sont pas rouverts individuellement). Si l'intention est de tout reprendre, l'utilisateur doit rouvrir chaque atelier concerné séparément, un par un, via son propre endpoint `rouvrir-atelierN`.

A5 --> A2 : cycle opérationnel partiel

Même mécanisme, avec `POST /api/v1/etudes/{id}/rouvrir-atelier2` → `Etude.RouvrirAtelier2()`. « Partiel » signifie méthodologiquement que seules les sources de risque et l'écosystème sont repris, sans revenir sur le cadrage (atelier 1). Là encore, ceci n'est pas une opération atomique dans le code : rouvrir l'atelier 2 ne touche à rien d'autre que `StatutAtelier2`. Rien n'empêche techniquement d'appeler `rouvrir-atelier2` puis `rouvrir-atelier1` dans n'importe quel ordre — le code n'impose aucune relation entre ces deux flèches pointillées, elles décrivent seulement deux points de ré-entrée possibles dans la méthode, choisis par l'utilisateur selon l'ampleur de la remise en cause.

Pourquoi ces flèches ne bloquent rien en aval

`Etude.RouvrirAtelierN()` ($N=1..4$) n'a qu'une seule garde : `StatutAtelierN == Validee` (sinon `InvalidOperationException`). Il n'y a **aucune vérification de l'état des ateliers suivants**. Concrètement, on peut rouvrir l'atelier 1 même si l'atelier 5 est déjà validé — le code le permet sans avertissement. Les rapports déjà générés pour les ateliers 2 à 5 restent inchangés tant que ces ateliers ne sont pas eux-mêmes revalidés (P16 : le reporting ne lit que des snapshots figés), mais rien dans le code ne signale à l'utilisateur que ses rapports téléchargés sont potentiellement obsolètes vis-à-vis du cadrage désormais modifié.

02b Diagramme de classes — chaque relation

27 relations composent ce diagramme. La notation UML utilisée mélange deux réalités techniques différentes qui se ressemblent visuellement mais ne le sont pas dans le code : la **composition** (losange plein, `*--`) y désigne tantôt une vraie entité EF Core « owned » (stockée dans une table séparée mais physiquement rattachée à son parent par une vraie contrainte de clé étrangère, jamais accessible indépendamment), tantôt un agrégat séparé relié par un simple `Guid` sans aucune contrainte de clé étrangère en base. Le tableau ci-dessous précise, pour chacune des 27 lignes du diagramme, laquelle des deux réalités s'applique.

NOTATION MERMAID	SENS UML	RÉALITÉ DANS LE CODE
<code>Etude ..> ServiceCreationSnapshot</code>	Dépendance	RÉEL Appel direct dans <code>Program.cs</code> : le endpoint <code>valider-atelierN</code> injecte <code>ServiceCreationSnapshotAtelierN</code> (une classe distincte par atelier — le diagramme en fait une seule boîte générique <code><x5></code>).
<code>ServiceCreationSnapshot ..> SnapshotAtelier</code>	Dépendance, crée	RÉEL <code>SnapshotAtelier.Creer(etudeId, numeroAtelier, version, contenuJson)</code> dans <code>Domain/Cadrage/SnapshotAtelier.cs</code> .
<code>SnapshotAtelier ..> ServiceGenerationRapport</code>	Dépendance	RÉEL <code>RapportAtelierNService.ConstruireAsync</code> lit exclusivement <code>ISnapshotAtelierRepository.ObtenirDernierParEtudeIdAsync</code> — jamais les agrégats vivants (principe P16).
<code>SnapshotAtelier "5" <.. RapportSyntheseGlobaleService</code>	Dépendance, cardinalité 5	RÉEL <code>RapportSyntheseGlobaleService.ConstruireAsync</code> appelle <code>ObtenirDernierParEtudeIdAsync</code> pour <code>numeroAtelier = 1, 2, 3, 4</code> et <code>5</code> successivement, et retourne <code>null</code> si l'un des cinq manque.

Etude "1" *-- "0..*" ValeurMetier : contient	Composition	AGRÉGAT SÉPARÉ	ValeurMetier a sa propre table valeurs_metier , son propre IValeurMetierRepository , et un simple Guid EtudeId indexé — aucune contrainte de clé étrangère en base (pas de HasOne/WithMany dans EbiosDbContext). La composition est un choix de modélisation du domaine (P8), pas une structure physique EF Core.
ValeurMetier "1..*" - "0..*" BienSupport : repose sur	Association simple	AGRÉGAT SÉPARÉ	Même schéma : BienSupport.ValeurMetierId est un Guid brut, sans contrainte FK. Le 1..* / 0..* (au lieu d'un losange) signale qu'un bien support n'est pas exclusif à une valeur métier — mais rien dans le code n'impose ce partage ni ne l'empêche : c'est purement déclaratif.
ValeurMetier "1" *-- "0..*" EvenementRedoute : redoute	Composition	AGRÉGAT SÉPARÉ	Même schéma que ci-dessus, table evenements_redoutes , ValeurMetierId sans FK.
Etude "1" *-- "0..1" SocleSecurite : applique	Composition	AGRÉGAT SÉPARÉ	SocleSecurite.EtudeId , index unique (garantissant le 0..1), pas de FK.
SocleSecurite "1" *-- "0..*" ReferentielApplicable : évalue	Composition	EF OWNED RÉEL	entity.OwnsMany(s => s.Referentiels, ...) avec WithOwner().HasForeignKey("SocleSecuriteId") — ici, la composition UML correspond exactement à une vraie clé étrangère en base, table referentiels_applicables .
Etude "1" *-- "0..*" CoupleSourceRisqueObjectifVise : identifie	Composition	AGRÉGAT SÉPARÉ	Table couples_sr_ov , EtudeId sans FK.
Etude "1" *-- "0..*" PartiePrenante : implique	Composition	AGRÉGAT SÉPARÉ	Table parties_prenantes , EtudeId sans FK.
PartiePrenante ..> ServiceCalculNiveauDangerosite	Dépendance	RÉEL	Appelé dans Program.cs , endpoint PUT .../parties-prenantes/{id}/dangerosite , avant PartiePrenante.EvaluerDangerosite(...) — le service calcule, l'entité stocke le résultat, jamais l'inverse.
CoupleSourceRisqueObjectifVise ..> ServiceCalculPertinence	Dépendance	RÉEL	Même schéma, Domain/SourcesRisque/ServiceCalculPertinence.cs .
Etude "1" *-- "0..*" ScenarioStrategique : projette	Composition	AGRÉGAT SÉPARÉ	Table scenarios_strategiques , EtudeId sans FK.
CoupleSourceRisqueObjectifVise "1" --> "1"	Association 1-1	AGRÉGAT SÉPARÉ, CONTRAINTE APPLICATIVE	ScenarioStrategique.CoupleSourceRisqueObjectifViseId , avec un

ScenarioStrategique : donne naissance à

index unique en base (`HasIndex(...).IsUnique()`) qui impose le "1" — mais toujours sans contrainte de clé étrangère référençant la table `couples_sr_ov`. Le endpoint `POST .../couples-sr-ov/{coupleId}/scenario-strategique` vérifie en plus, en code, qu'aucun scénario n'existe déjà pour ce couple avant d'en créer un (`ObtenirParCoupleIdAsync`).

EvenementRedoute "0..1" --> "0..*" Association

ScenarioStrategique : est ciblé par

AGRÉGAT SÉPARÉ `ScenarioStrategique.EventementRedouteId`, Guid brut. Le "0..1" côté `EvenementRedoute` est mal formé au sens strict (un ER peut être ciblé par plusieurs scénarios, donc ce devrait être "1" côté ER vu depuis un scénario donné) — à lire comme « chaque scénario référence exactement un ER, un ER peut n'être référencé par aucun scénario ».

ScenarioStrategique "1" *-- "1..*" Composition

CheminAttaque : se décline en

AGRÉGAT SÉPARÉ Table `chemins_attaque`, `ScenarioStrategiqueId` sans FK. Le `1..*` (au moins un chemin) n'est **pas vérifié par le code** : rien n'empêche un scénario stratégique d'exister sans aucun chemin d'attaque ; c'est `ServiceValidationCompletudeAtelier3` qui, au moment de valider l'atelier, vérifie une condition proche (au moins un chemin sur l'ensemble de l'étude, pas par scénario).

CheminAttaque "0..*" o-- "0..*" Agrégation

PartiePrenante : traverse

PAS DE TABLE DE LIAISON Le losange creux (agrégation, relation plus faible que la composition) n'a **aucune traduction directe** : il n'existe pas de relation many-to-many `CheminAttaque` ↔ `PartiePrenante` en base. Le lien réel passe uniquement par la table intermédiaire `evenements_intermediaires` (ligne suivante) : une partie prenante n'est associée à un chemin qu'au travers d'un `EvenementIntermediaire` nommé.

CheminAttaque "1" *-- "0..*" Composition

EvenementIntermediaire : génère

EF OWNED RÉEL `entity.OwnsMany(c => c.EventementsIntermediaires, ...)`, `HasForeignKey("CheminAttaqueId")`, table `evenements_intermediaires`.

EvenementIntermediaire "1" --> "1" Association

PartiePrenante : concerne

GUID BRUT DANS UNE ENTITÉ OWNED `EvenementIntermediaire.PartiePrenanteId` est un Guid sans navigation ni FK vers `parties_prenantes`, alors même que l'entité qui le porte est, elle, physiquement owned par `CheminAttaque`.

PartiePrenante "1" *- - "0..*" Composition

MesureEcosysteme : réduit sa dangerosité via

EF OWNED RÉEL `entity.OwnsMany(p => p.Mesures, ...)`, `HasForeignKey("PartiePrenanteId")`, table `mesures_ecosysteme`. Ajoutée via `POST .../parties-prenantes/{id}/mesures` — visible et modifiable uniquement dans la page de l'**Atelier 3** du frontend (`MesuresEcosystemeSection` dans `AtelierPage.tsx`), pas dans l'Atelier 2, bien que la `PartiePrenante` elle-même soit créée en Atelier 2.

ScenarioOperationnel "1" *-- "1..*" ModeOperatoire : compose	Composition	EF OWNED RÉEL entity.OwnsMany(s => s.ModesOperatoires, ...), HasForeignKey("ScenarioOperationnelId"), table modes_operatoires. Le 1..* n'est, ici non plus, pas vérifié à la création de ScenarioOperationnel (qui peut naître sans aucun mode) ; il est vérifié seulement par ServiceValidationCompletudeAtelier4 à la validation de l'atelier.
ModeOperatoire "1" *- - "1..*" ActionElementaire : décompose en	Composition	EF OWNED RÉEL, IMBRIQUÉ mo.OwnsMany(m => m.ActionsElementaires, ...) à l'intérieur du OwnsMany précédent — deux niveaux d'imbrication, table actions_elementaires. Ce 1..* -là est vérifié immédiatement : ModeOperatoire.Creer lève une ArgumentException si la liste d'actions est vide.
ActionElementaire "0..*" --> "1" BienSupport : cible	Association	GUID BRUT ActionElementaire.BienSupportId, sans FK vers biens_support — mais vérifié en code applicatif : Program.cs (fonction ParseActionsElementairesAsync) interroge IBienSupportRepository pour confirmer que le bien existe et appartient à la même étude avant d'accepter l'action.
ScenarioOperationnel ..> ServiceCalculVraisemblance	Dépendance	RÉEL Utilisé indirectement via la propriété calculée ModeOperatoire.VraisemblanceCalculee (Domain/ScenariosDeRisque/ScenarioOperationnel.cs), pas stockée.
ScenarioDeRisque "1" *-- "1" CheminAttaque : repose sur	Composition	AGRÉGAT SÉPARÉ, INDEX UNIQUE ScenarioDeRisque.CheminAttaqueId, HasIndex(...).IsUnique(), sans FK.
ScenarioDeRisque "1" *-- "0..1" ScenarioOperationnel : repose sur	Composition	AUCUNE COLONNE Cette relation n'a aucune traduction directe en base : ScenarioDeRisque ne stocke aucun ScenarioOperationnelId. Le lien passe uniquement par le CheminAttaqueId commun : ServiceAssemblageScenariosDeRisque retrouve le ScenarioOperationnel correspondant en appelant IScenarioOperationnelRepository.ObtenirParCheminIdAsync(cheminAttaqueId) à chaque lecture, jamais par une colonne stockée.
ScenarioDeRisque ..> ServiceCalculNiveauRisque	Dépendance	RÉEL Domain/ScenariosDeRisque/ServiceCalculNiveauRisque.cs, appelé à la fois pour le niveau initial (dans ServiceAssemblageScenariosDeRisque) et pour le niveau résiduel (endpoint PUT ../risque-residuel).
Etude "1" *-- "0..1" PlanTraitementRisque : décide	Composition	AGRÉGAT SÉPARÉ PlanTraitementRisque.EtudeId, index unique, pas de FK.
PlanTraitementRisque "1" *-- "0..*"	Composition	EF OWNED RÉEL entity.OwnsMany(p => p.Mesures, ...), HasForeignKey("PlanTraitementRisqueId"), table

MesureTraitementRisque
e : planifie

mesures_traitement_risque .

MesureTraitementRisque Association
e "0..*" --> "1..*" many-to-
ScenarioDeRisque : many
traite

COLONNE TABLEAU, SANS FK

MesureTraitementRisque.ScenariosDeRisqueIds est une `List<Guid>` mappée via `PrimitiveCollection` vers une colonne Postgres native `uuid[]` (`scenarios_de_risque_ids`) — pas une table de jointure, pas de contrainte référentielle. L'intégrité (chaque id référencé existe et appartient à l'étude) est vérifiée uniquement à l'écriture, par `Program.cs` (fonction `ValiderMesureTraitementRisqueAsync`), jamais par la base.

Ce que cela signifie globalement

Sur les 27 relations, **6 seulement** correspondent à une contrainte de clé étrangère réelle en base (les entités EF `OwnsMany`). Les **19 autres** composition/association du diagramme représentent des relations de domaine (P8, DDD) volontairement non contraintes par la base de données — l'intégrité référentielle est reconstruite à la main dans `Program.cs`, notamment via les blocs de suppression en cascade (par exemple `DELETE .../couples-sr-ov/{id}`, qui supprime explicitement le scénario stratégique, les chemins, les scénarios opérationnels et les scénarios de risque associés, un par un, en code). **2 relations** (`CheminAttaque o-- PartiePrenante` et `ScenarioDeRisque *-- ScenarioOperationnel`) n'ont aucune traduction directe : ce sont des relations dérivées, recalculées à la demande à partir d'autres colonnes, jamais stockées telles quelles.

03 Diagramme de composants — chaque bloc

RÉEL **FE — Frontend SPA.** Répertoire `frontend/src/`. Chaque appel réseau part de

`frontend/src/lib/api.ts`, fonction `apiFetch(path, options)`, qui construit une requête `fetch()` standard vers `http://localhost:5197/api/v1` (constante `API_BASE`). Il n'y a pas de client HTTP généré ni de bibliothèque tierce (pas d'Axios, pas de React Query) : chaque fonction exportée de `api.ts` (une soixantaine) enveloppe un seul appel `apiFetch`.

NON IMPLÉMENTÉ « Endpoints » et « Authentification » (composant Présentation). Il n'existe **aucune**

classe nommée « Endpoint » ou « Présentation » dans le code. Les 96 routes HTTP sont déclarées comme 96 appels directs

`app.MapGet(...)` / `app.MapPost(...)` / `app.MapPut(...)` / `app.MapDelete(...)` à la racine d'un seul fichier, `src/EbiosRM.Api/Program.cs` (2069 lignes), selon le style « Minimal API » d'ASP.NET Core — pas de contrôleurs, pas d'attributs de routage. Le sous-composant « Authentification » ne correspond à rien : `Program.cs` ne contient ni `UseAuthentication()`, ni `UseAuthorization()`, ni attribut `[Authorize]`, ni schéma d'identité configuré.

NON IMPLÉMENTÉ (COUCHE FICTIVE) « **Application** » / **Handlers de cas d'usage**. Cette couche n'existe pas physiquement : il n'y a pas de dossier `Modules/Application`, pas de classe `*Handler`, pas de médiateur (pas de MediatR ni équivalent). Ce que le diagramme dessine comme un saut « Endpoint → Handler → Agrégat → Services » se déroule en réalité *dans le corps d'une seule fonction lambda*, celle passée en second argument à `app.MapPost`. Par exemple, pour `valider-atelier5` (voir §04), les quatre étapes « recevoir la requête », « vérifier la complétude », « muter l'agrégat » et « créer le snapshot » sont quatre lignes consécutives de la même fonction anonyme dans `Program.cs`, pas quatre composants séparés qui se transmettent la main.

RÉEL **Core Engine — Domain**. Répertoire `src/EbiosRM.Api/Modules/CoreEngine/Domain/`, organisé en cinq sous-dossiers par atelier (`Cadrage`, `SourcesRisque`, `ScenariosDeRisque` — qui contient à la fois les classes de l'atelier 3, 4 et 5). « Etude — agrégat racine » désigne littéralement la classe `Etude` dans `Domain/Cadrage/Etude.cs`. « Services de domaine » désigne l'ensemble des classes statiques `ServiceCalcul*`, `ServiceCreationSnapshot*` et `ServiceValidationCompleteness*` — une quinzaine de classes, pas un composant unique.

RÉEL **Infrastructure — Repositories EF Core**. Répertoire `Modules/CoreEngine/Infrastructure/`, une classe `*Repository` par agrégat (`EtudeRepository`, `ValeurMetierRepository`, ..., seize classes), chacune implémentant une interface `I*Repository` déclarée à côté de l'agrégat correspondant dans `Domain/`. Le mapping EF Core lui-même (tables, colonnes, `OwnsMany`) est centralisé dans `Infrastructure/Persistence/EbiosDbContext.cs`, pas dans les repositories.

RÉEL **Reporting — Génération de rapports**. Répertoire `Modules/Reporting/` : pour chaque atelier N (1 à 5), une paire `RapportAtelierNData.cs` (DTO), `RapportAtelierNService.cs` (lit le snapshot, projette vers le DTO) et `RapportAtelierNPdfGenerator.cs` (rendu PDF via la bibliothèque QuestPDF). S'y ajoutent `RapportSyntheseGlobaleData/Service/PdfGenerator.cs` pour le document transverse. La flèche EP --> RSVC --> DB du diagramme signifie que l'endpoint de rapport interroge directement `ISnapshotAtelierRepository` (donc la base), sans passer par un agrégat vivant.

NON IMPLÉMENTÉ **Assistance IA — Suggestions**. Aucun code ne correspond à ce composant. Aucun dossier `Modules/IA`, aucune dépendance vers un fournisseur de modèle de langage dans `EbiosRM.Api.csproj`. C'est un composant cible, dessiné pour fixer par avance la frontière architecturale qu'il devra respecter le jour où il sera construit (n'entrer que par des commandes explicites, jamais d'écriture directe au Domain ou à la base) — cf. `PROJECT_CONTEXT.md`, « Slice 7 — Assistant IA (non commencé) ».

RÉEL **PostgreSQL**. Base `ebiosrm` (développement) / `ebiosrm_test` (intégration), schéma `core_engine`, connectée via le pilote `Npgsql` (`options.UseNpgsql(...)` dans `Program.cs`).

Ce diagramme illustre `valider-atelierN` de façon générique. La route dessinée, `POST /etudes/{id}/ateliers/{numero}/valider`, est une **simplification** : la route réelle n'a pas de paramètre `{numero}`, elle est répétée telle quelle cinq fois, une par atelier — `POST /api/v1/etudes/{id:guid}/valider-atelier1` jusqu'à `valider-atelier5`, chacune codée comme un bloc `app.MapPost` séparé dans `Program.cs` (aucune route générique paramétrée par un numéro n'existe). De même, les participants `ValiderAtelierHandler`, `ServiceValidationCompletude` et `ServiceCreationSnapshot` désignent chacun cinq classes concrètes différentes (une par atelier) — le diagramme les présente comme un seul acteur générique pour rester lisible.

- **1–2.** Clic puis `POST` — **RÉEL**, réalisé côté frontend par la fonction `validerAtelier5(etudeId)` de `lib/api.ts`, appelée depuis `handleValiderAtelier5` dans `AtelierPage.tsx`.
- **3.** `EP → H : Handle(Commande)` — **FICTIF** il n'y a pas de passage de message entre deux objets : c'est l'entrée dans le corps de la lambda `app.MapPost("/api/v1/etudes/{id:guid}/valider-atelier5", async (Guid id, IEtudeRepository repo, ServiceValidationCompletudeAtelier5 serviceValidation, ServiceCreationSnapshotAtelier5 serviceSnapshot, EbiosDbContext db, CancellationToken ct) => { ... })`. Les paramètres de la lambda sont résolus par injection de dépendances ASP.NET Core, pas transmis par un « Endpoint » qui les collecterait puis les passerait à un « Handler ».
- **4–7. Obtenir l'étude** — **RÉEL** `await repo.ObtenirParIdAsync(id, ct)`, qui exécute une requête EF Core `SELECT` avec ses `Include` nécessaires selon le repository. Retourne `null → 404` si l'étude n'existe pas.
- **8–9. Contrôle de complétude** — **RÉEL** `await serviceValidation.VerifierAsync(id, ct)` retourne un `ResultatValidationCompletude(bool EstComplet, List<string> ElementsManquants)`. Si `EstComplet` est faux, l'endpoint retourne `400 BadRequest` immédiatement — aucune transaction n'a encore été ouverte, aucune écriture n'a eu lieu. Pour l'atelier 5, `ServiceValidationCompletudeAtelier5.VerifierAsync` exige qu'au moins un `ScenarioDeRisque` existe et que chacun ait un risque résiduel évalué.
- **10–11. Muter l'agrégat** — **RÉEL** `etude.ValiderAtelier5()` (méthode de `Etude.cs`) vérifie l'invariant `StatutAtelier5 == EnCours` (sinon `InvalidOperationException`, capturée plus loin et transformée en `400`), puis passe `StatutAtelier5` à `Validee`. Cette mutation se produit en mémoire, rien n'est encore écrit en base.
- **12–13. Créer le snapshot** — **RÉEL** `await serviceSnapshot.CreerAsync(id, ct)` relit l'ensemble des agrégats vivants concernés (via `ServiceAssemblageScenariosDeRisque` pour l'atelier 5), sérialise en JSON, appelle `SnapshotAtelier.Creer(...)` et persiste immédiatement via `ISnapshotAtelierRepository.AjouterAsync` — cet appel fait sa **propre** écriture en base, avant même l'étape suivante.
- **14. Note « transaction unique »** — **RÉEL, MAIS IMPRÉCIS SUR LE DIAGRAMME** la transaction est ouverte *avant* l'étape 10, pas après l'étape 13 comme le laisse penser sa position sur le schéma : `await using var transaction = await db.Database.BeginTransactionAsync(ct);` est appelé juste après le

contrôle de complétude (étape 9), et englobe la mutation de l'agrégat (10), l'appel `repo.MettreAJourAsync` et la création du snapshot (12) dans un seul bloc `try`. Une exception à n'importe laquelle de ces étapes déclenche `transaction.RollbackAsync()`.

- **15–19. Persister et répondre** — **RÉEL** `await repo.MettreAJourAsync(etude, ct)` (un simple `SaveChangesAsync`, l'entité étant déjà suivie par le contexte), puis `await transaction.CommitAsync(ct)`, puis `Results.Ok(new { etude, snapshotVersion = snapshot.Version })` — le corps JSON réel de la réponse 200, pas seulement « OK ».

05 Diagrammes d'états — chaque transition

1. Cycle générique d'un atelier

Les libellés du diagramme (`NonDemarre`, `Valide`) sont des renommages lisibles ; les valeurs réelles de l'énumération `StatutEtude` (déclarée une seule fois dans `Etude.cs`) sont `Brouillon`, `EnCours`, `Validee`. Cette même énumération est réutilisée pour cinq propriétés indépendantes de la classe `Etude` : `Statut` (atelier 1), `StatutAtelier2`, `StatutAtelier3`, `StatutAtelier4`, `StatutAtelier5` — cinq colonnes distinctes dans la table `etudes`, pas une collection. Les transitions correspondent chacune à une méthode réelle (`DemarrerAtelierN()`, `ValiderAtelierN()`, `RouvrirAtelierN()`) répétée cinq fois avec un garde-fou identique (vérification du statut courant, sinon `InvalidOperationException`).

2. Statut global de l'étude

NON IMPLÉMENTÉ Ce diagramme entier ne correspond à **aucun code existant**. `Etude` n'expose ni champ `Finalisee`, ni propriété calculée équivalente, ni aucune méthode qui agrégerait les cinq statuts d'atelier en un statut global unique. Une recherche du terme `Finalisee` dans l'ensemble du dépôt (backend et frontend) ne retourne aucun résultat. Pour savoir si une étude a franchi les cinq ateliers, le code existant (`RapportSyntheseGlobaleService.ConstruireAsync`) vérifie les cinq statuts un par un en tentant de charger les cinq derniers snapshots et en retournant `null` si l'un manque — il n'y a pas de raccourci « l'étude est finalisée ». Ce diagramme documente une simplification conceptuelle utile pour raisonner sur la méthode EBIOS RM, pas une classe ou un champ du logiciel.

3. Suivi d'une mesure du plan de traitement du risque

RÉEL Les trois états correspondent exactement à l'énumération `StatutMesure` (`Domain/ScenariosDeRisque/PlanTraitementRisque.cs`) : `ALancer`, `EnCours`, `Termine`. Contrairement aux deux diagrammes précédents, il n'existe **aucune méthode de transition** dans le code : `MesureTraitementRisque` n'a pas de méthode

`DemarrerMiseEnOeuvre()` ou `Terminer()`. Le champ `Statut` est un simple paramètre libre de `PlanTraitementRisque.AjouterMesure(...)` et `ModifierMesure(...)` : l'analyste choisit directement la valeur dans un menu déroulant du frontend (`LIBELLE_STATUT_MESURE` dans `AtelierPage.tsx`), sans qu'aucune règle de code n'empêche par exemple de passer directement de `ALancer` à `Termine`. Les flèches de ce diagramme ne sont donc pas des gardes appliquées par le domaine, contrairement au premier diagramme d'état de cette section.

06 Diagramme de déploiement

RÉEL Trois nœuds, conformes à l'environnement de développement actuel : le navigateur charge le bundle produit par `vite build` (frontend servi séparément, port 5174 en développement) ; `EbiosRM.Api` tourne sous Kestrel (serveur HTTP intégré à ASP.NET Core, aucun IIS ni Nginx en amont dans cet environnement), port 5197 ; PostgreSQL 16 écoute sur le port 5433 de la même machine dans cet environnement de développement (voir `appsettings.Development.json`). La connexion base de données passe par `Npgsql`, le fournisseur ADO.NET pour PostgreSQL référencé dans `EbiosRM.Api.csproj`. Aucune configuration de déploiement de production (conteneurisation de l'API, reverse proxy, certificats TLS) n'existe dans le dépôt à ce jour ; seul `docker-compose` pour PostgreSQL est présent.

Document compagnon de `EBIOS-RM-diagrammes-UML.pdf`. Les tags **RÉEL** / **PARTIEL OU HORS BASE** /

NON IMPLÉMENTÉ reflètent l'état du code au moment de la rédaction — à revérifier si le code évolue.